

codehive

From F-Strings to the format() Method

Volume VIII: String Engineering

1. The Evolution of Formatting

String formatting is a critical skill for any Python developer. It allows you to inject dynamic data into static text efficiently.

Historical Context

- **Pre-Python 3.6:** Developers relied on the % operator and `.format()`.
- **Python 3.6+:** F-Strings (Formatted String Literals) were introduced.

F-Strings are now the preferred method because they are faster, more readable, and more concise.

```
# The simplest f-string  
txt = f"The price is 49 dollars"  
print(txt)
```

2. Placeholders and Modifiers

To inject a variable into an f-string, you use **placeholders**: curly brackets {}.

Using Modifiers

A modifier is used to format the value inside the placeholder. You add a colon : followed by a formatting type.

```
price = 59
# .2f modifier ensures 2 decimal places
txt = f"The price is {price:.2f} dollars"
print(txt) # Output: The price is 59.00 dollars
```

Modifiers can be used on literals as well: `f"{95: .2f}"`

3. Expressions & Logic in F-Strings

F-Strings are not just for variables; they can evaluate **expressions** directly.

Math Operations

```
txt = f"Total: {20 * 59}"
```

Ternary Operations (If...Else)

You can perform conditional logic inside the string:

```
price = 49
status = f"It is {'Expensive' if price > 50 else 'Cheap'}"
print(status)
```

4. Executing Functions

You can call built-in methods or custom functions inside placeholders.

Built-in Methods

```
fruit = "apples"  
print(f"I love {fruit.upper()}") # I LOVE APPLES
```

Custom Functions

```
def to_meters(feet):  
    return feet * 0.3048  
  
alt = 30000  
print(f"Flight altitude: {to_meters(alt)}m")
```

5. Alignment & Thousand Separators

F-strings make UI output formatting simple with alignment and numbering modifiers.

Symbol	Meaning	Example
:,	Thousand separator	{59000:,} -> 59,000
:<	Left Align	{val:<10}
:>	Right Align	{val:>10}
:^	Center Align	{val:^10}

Pro Tip: Thousand separators significantly improve the readability of financial data.

6. Formatting Types Reference

A quick reference list of legal formatting types in Python:

:b Binary format

:d Decimal format

:e Scientific (lower e)

:f Fixed point number

:x Hex format (lower)

:o Octal format

:% Percentage format

Note: The **:%** modifier automatically multiplies the value by 100 and adds the % sign.

7. The format() Method (Legacy)

While f-strings are preferred, the `.format()` method is still widely used in older codebases.

```
price = 49
txt = "The price is {} dollars"
print(txt.format(price))
```

Placeholders with Modifiers

The syntax for modifiers is identical in `.format()`:

```
txt = "Price: {:.2f}"
print(txt.format(49))
```


8. Advanced Indexing in format()

The `.format()` method offers powerful indexing capabilities for re-using variables.

Numbered Indexes

```
age = 36
name = "John"
txt = "{1} is {0} years old. {1} likes Python."
print(txt.format(age, name))
```

Named Indexes

```
txt = "I have a {car}, it's a {model}."
print(txt.format(car="Ford", model="Mustang"))
```

9. Practical Exercise

Knowledge Check

Question: What placeholders are used when dealing with f-strings?

1. []
2. ()
3. {}

Final Summary

Use **F-Strings** for performance and clean code. Keep **.format()** in your toolkit for legacy support and complex positional indexing.

10. Completion Certificate

CERTIFIED BY

codehive

This document verifies completion of the Python String
Formatting Tutorial Series.

STR-2024-FORMAT